

TÜRKİYE CUMHURİYETİ
YILDIZ TEKNİK ÜNİVERSİTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



OTONOM AJAN YAKLAŞIMIYLA KOD VERİ
TABANINDAN EĞİTİM İÇERİĞİ ÜRETEEN YAPAY ZEKA
TABANLI YAZILIMIN TASARIMI VE GELİŞTİRMESİ

21011035 – BORA İLCİ
21011506 – KAAN EMRE KARA

BİLGİSAYAR PROJESİ

Danışman
Prof. Dr. Mehmet Sıddık AKTAŞ

Ocak, 2026

TEŞEKKÜR

Öncelikle bu projenin gerçekleştirilmesinde değerli katkılarını esirgemeyen danışmanımız Prof. Dr. Mehmet Sıddık AKTAŞ’a sonsuz teşekkürlerimizi sunarız. Kendisinin rehberliği, akademik desteği ve motivasyonu olmaksızın bu çalışmanın tamamlanması mümkün olamazdı.

Ailelerimize; eğitim hayatımız boyunca gösterdikleri sabır, anlayış ve koşulsuz destek için minnetimizi ifade etmek isteriz. Ayrıca bu süreçte yanımızda olan arkadaşlarımıza ve bize ilham veren tüm akademisyenlere teşekkür ederiz.

Son olarak, Yıldız Teknik Üniversitesi Bilgisayar Mühendisliği Bölümü’ne sağladığı akademik ortam ve imkânlar için şükranlarımızı sunarız.

BORA İLCİ
KAAN EMRE KARA

İÇİNDEKİLER

KISALTMA LİSTESİ	v
ŞEKİL LİSTESİ	vi
TABLO LİSTESİ	vii
ÖZET	viii
ABSTRACT	x
1 GİRİŞ	1
1.1 Problem ve Motivasyon	1
1.2 Ön İnceleme	3
2 SİSTEM ANALİZİ VE FİZİBİLİTE	4
2.1 Sistem Analizi	4
2.2 Gereksinim Analizi	6
2.3 Bilgi Tabanı Sistemi	8
2.3.1 Amaç ve Motivasyon	8
2.3.2 Üretim Süreci	8
2.3.3 Dosya Yapısı ve İçerik	9
2.3.4 Eğitim Hattıyla Entegrasyon	9
2.3.5 Eğitim İçeriği Üretim Süreci	9
2.3.6 Ampirik Gözlemler	11
2.4 Fizibilite	12
2.4.1 Teknik Fizibilite	12
2.4.2 Yasal Fizibilite	13
2.4.3 Ekonomik Fizibilite	14
2.4.4 İşgücü ve Zaman Fizibilitesi	15
3 SİSTEM TASARIMI	16
3.1 Arayüz Tasarımı	16
3.1.1 Komut Satırı Arayüzü	16
3.1.2 Web Tabanlı Kontrol Paneli	16

3.2	Veri Yapısı Tasarımı	17
3.2.1	Yapılandırma Yönetimi	17
3.2.2	İzole Çalışma Alanları (Kısıtlı Dosya Sistemi)	18
3.3	Nesneye Yönelik Tasarım ve Ajan Mimarisi	18
3.3.1	Dinamik Ajan Yapısı	18
3.3.2	Eşgüdüm Katmanı	19
3.3.3	Model Geçit Katmanı (LLM Gateway)	19
3.3.4	Araç Takımları	20
3.3.5	Bilgi Tabanı Şeması	20
3.3.6	Gözlemlenebilirlik ve Ölçüm	21
4	UYGULAMA SONUÇLARI ve TESTLER	22
4.1	Uygulama Tanıtımı	22
4.2	Test Sonuçları	23
4.2.1	Çalışma Süresi ve Araç Kullanımı (Metrik Özeti)	23
4.2.2	Token Kullanımı	24
4.3	Doğrulama	24
4.3.1	LLM-yargıç Değerlendirmesi	24
4.3.2	Uzman Değerlendirmesi (İnsan Tercihi)	26
4.3.3	Gereksinim Uyumluluk Analizi	26
5	SONUÇ VE TARTIŞMA	28
5.1	Elde Edilen Kazanımlar	28
5.2	Kısıtlar ve Maliyet	29
5.3	Gelecek Çalışmalar	29
A	YARGICI DEĞERLENDİRME ÖRNEKLERİ	31
A.1	Yargıç İstemi (Judge Prompt)	31
A.2	Örnek Eğitim İçeriği Karşılaştırması	31
A.2.1	Baseline Çıktısı (Özet)	31
A.2.2	Deep Agent Çıktısı (Özet)	32
A.3	Yargıç Değerlendirme Çıktısı (Örnek)	32
	Referanslar	33
	Özgeçmiş	34

KISALTMA LİSTESİ

LLM Large Language Model (Büyük Dil Modeli)

RAG Retrieval-Augmented Generation (Geri Getirme Artırılmış Üretim)

API Application Programming Interface (Uygulama Programlama Arayüzü)

CLI Command Line Interface (Komut Satırı Arayüzü)

SDLC Software Development Life Cycle (Yazılım Geliştirme Yaşam Döngüsü)

MIT Massachusetts Institute of Technology License

ŞEKİL LİSTESİ

Şekil 2.1	Deep Agent mimarisi: üst düzey görünüm	5
Şekil 2.2	Sistem veri akış diyagramı.	7
Şekil 2.3	Eğitim içeriği üretim süreci akış diyagramı.	10
Şekil 2.4	İşgücü zaman çizelgesi	15
Şekil 3.1	Kullanıcı etkileşimi ve veri akış diyagramı.	17
Şekil 3.2	Eşgüdüm katmanı ve dinamik bileşim	19

TABLO LİSTESİ

Tablo 2.1	Bilgi tabanı bileşenleri ve tipik boyutları.	9
Tablo 2.2	Donanım gereksinimleri.	12
Tablo 4.1	Metrik özeti (Deep Agent)	23
Tablo 4.2	Metrik özeti (Baseline)	24
Tablo 4.3	LLM-yargıç özet puanları	25

OTONOM AJAN YAKLAŞIMIYLA KOD VERİ TABANINDAN EĞİTİM İÇERİĞİ ÜRETEYEN YAPAY ZEKA TABANLI YAZILIMIN TASARIMI VE GELİŞTİRMESİ

BORA İLCİ
KAAN EMRE KARA

Bilgisayar Mühendisliği Bölümü
Bilgisayar Projesi

Danışman: Prof. Dr. Mehmet Sıddık AKTAŞ

Bu proje, büyük ve karmaşık kod tabanlarını anlama ve dokümantasyon üretme zorluğunu, yüksek kaliteli eğitim içeriği oluşturabilen otonom bir yapay zekâ ajan sistemi geliştirerek çözmeyi amaçlamaktadır. Tek ajanlı yaklaşımlar, Büyük Dil Modellerinin (LLM) sınırlı bağlam penceresi nedeniyle geniş ölçekli projelerde yeterli kapsama ve doğruluk sağlayamamaktadır. Bu kısıtlamayı aşmak için, alt ajan metodolojisine dayalı özgün bir *Deep Agent* mimarisi geliştirilmiştir. Bu mimari, geleneksel tek ajanlı sistemlerden farklı olarak, yönetici ajanın görevi daha küçük parçalara ayırıp bağımsız alt-ajanlara devretmesi ve her alt-ajanın yalnızca kendi kapsamındaki dosyalarla sınırlı, izole bir çalışma alanında faaliyet göstermesi prensibine dayanır; böylece bağlam penceresi sınırı aşılmadan büyük ölçekli projeler güvenli biçimde analiz edilebilir.

Önerilen sistem iki aşamada çalışır. Birinci aşamada yönetici ajan, yazılım projesini bağımsız parçalara ayırarak her bir parça için alt ajanlara analiz görevleri atar; bu çıktılar birleştirilerek yeniden kullanılabilir, yapılandırılmış bir ara özet temsili oluşturulur. İkinci aşamada öğretici doküman üretici, bu ara temsili birincil başvuru kaynağı olarak kullanarak kapsamlı ve adım adım ilerleyen öğrenme materyalleri üretir. Yöntem, beş farklı açık kaynak yazılım projesi üzerinde değerlendirilmiş; ayrıca değerlendirme örnekleri arasına, kullanılan modelin bilgi kesim tarihinden sonra oluşturulmuş yazılım projeleri de dahil edilmiştir. Önerilen yöntem, ara özet temsili oluşturmada doğrudan üretim yapan temel yaklaşımla karşılaştırılmıştır. Dört farklı LLM'in yargıç olarak

kullanıldığı deęerlendirmeler (LLM-as-a-Judge; Claude 4.5 Sonnet, GPT-OSS-120B, Gemini 2.5 Pro ve Grok Code Fast 1), *Deep Agent* mimarisinin temel yaklaşıma kıyasla doęrulukta

Anahtar Kelimeler: Derin Ajan Mimarisi, Çok Ajanlı Sistemler, Ara Özet Temsili, Öğretici Doküman Üretimi, Büyük Dil Modelleri, Bağlam Penceresi Yönetimi, Eşgüdüm, Alt Ajan Metodolojisi

ABSTRACT

DESIGN AND DEVELOPMENT OF AN ARTIFICIAL INTELLIGENCE-BASED SOFTWARE THAT GENERATES EDUCATIONAL CONTENT FROM A CODE DATABASE USING AN AUTONOMOUS AGENT APPROACH

BORA İLCİ
KAAN EMRE KARA

Department of Computer Engineering
Computer Project

Advisor: Prof. Dr. Mehmet Siddık AKTAŞ

This project addresses the challenge of understanding and documenting large, unfamiliar codebases by developing an autonomous artificial intelligence agent system that generates high-quality educational content. Traditional single-agent approaches often fail on large-scale projects due to the finite context window of Large Language Models (LLMs). To overcome this limitation, we implement a novel *Deep Agent* architecture based on a sub-agent methodology. Unlike traditional single-agent systems, this architecture employs a hierarchical delegation strategy where the supervisor agent decomposes tasks into smaller units assigned to sub-agents, each operating within an isolated workspace limited to its assigned scope—thereby circumventing context window limitations while maintaining traceability and security.

The system operates in two phases. In the first phase, a supervisor agent coordinates multiple sub-agents to analyze isolated parts of a software project; their findings are consolidated into a structured intermediate summary (a reusable representation that captures key components and relationships). In the second phase, an instructional document generator uses this intermediate summary to produce comprehensive, step-by-step learning materials. The proposed system was evaluated on **five real-world open-source software projects**, including projects created after the underlying model’s knowledge cutoff date, and compared against a baseline approach that performs direct generation without an intermediate summary. Results from LLM-as-a-Judge experiments

(where four LLMs were used as judges: Claude 4.5 Sonnet, GPT-OSS-120B, Gemini 2.5 Pro, and Grok Code Fast 1) show that the *Deep Agent* architecture substantially improves fidelity, pedagogy, and coverage, effectively transforming complex software into actionable learning resources.

Keywords: Deep Agent Architecture, Multi-Agent Systems, Intermediate Summarization, Instructional Document Generation, Large Language Models, Context Window Management, Orchestration, Sub-Agent Methodology, Software Documentation

Modern yapay zekâ uygulamalarının temelinde, büyük metin ve kod veri kümeleri üzerinde eğitilmiş büyük dil modelleri (LLM) yer alır. Bu modeller, kendilerine sunulan bilgiyi anlama ve üretme konusunda yüksek kapasiteye sahip olsalar da; uzun soluklu planlama, araç kullanımı ve çok adımlı görevlerde tek başına çalıştırıldıklarında güvenilirlik ve süreklilik açısından sınırlılıklar gösterebilir.

Bu noktada ajan tabanlı sistemler devreye girmektedir [1]. Bir ajan, LLM'yi karar verme çekirdeği olarak kullanır; hedefe yönelik planlama yapar, dosya okuma gibi araçları kontrollü biçimde etkinleştirir ve süreç boyunca elde edilen bulgulara göre stratejisini günceller.

Bu proje, yazılım mühendisliğinde özellikle yeni geliştiricilerin büyük kod tabanlarını (*codebase*) kısa sürede anlamasını kolaylaştıracak bir otomatikleştirme yaklaşımına odaklanır. Amaç, otonom ajan mimarisi kullanarak bir kod tabanını sistematik biçimde incelemek, kalıcı bir *bilgi tabanı* (*knowledge base*) üretmek ve bu birikimi pedagojik değeri yüksek eğitim içeriklerine dönüştürmektir.

1.1 Problem ve Motivasyon

Yeni bir geliştiricinin, binlerce dosyadan oluşan yabancı bir yazılım sistemine uyum sağlaması yüksek maliyetli ve zaman alan bir süreçtir. Dokümantasyonun eksik veya güncel olmadığı durumlarda, sistem mimarisinin, veri akışlarının ve kritik bileşenlerin anlaşılması haftalar sürebilmektedir.

Örneğin finansal teknoloji alanında çalışan bir yazılım mühendisinin, on binlerce dosyadan oluşan yeni bir ödeme altyapısı üzerinde değişiklik yapması gerektiğinde; mimariyi anlaması, iş akışlarını kavraması ve küçük bir değişikliği güvenle uygulayabilmesi çoğu zaman uzun bir hazırlık evresi gerektirir. İnsan gözetiminde yürüyen bu uyum sağlama süreci, yapılandırılmış bilgi çıkarımını otomatikleştiren araçlar olmadığında daha da uzamakta; iş devri ve bakım maliyetleri artmaktadır.

Bu süreçte LLM tabanlı yaklaşımlar umut verici olsa da, pratikte *bağlam penceresi* (*context window*) olarak adlandırılan teknik sınıra takılmaktadır. Bir model aynı anda yalnızca sınırlı miktarda bilgiyi bağlamında taşıyabildiğinden, büyük bir kod tabanını tek seferde değerlendirmek odak kaybına, maliyet artışına ve hatalı çıkarımlara neden olabilir. Dolayısıyla ölçeklenebilir çözüm, kod tabanını anlamlı parçalara ayıran ve üretilen ara çıktıları daha sonra birleştiren bir eşgüdüm (*orchestration*) yaklaşımı gerektirir.

Ajan tabanlı sistemler, yazılım geliştirme yaşam döngüsünün birçok aşamasını otomatikleştirme potansiyeline sahiptir. Güncel uygulama alanları arasında şunlar bulunmaktadır:

- **Otomatik hata ayıklama:** Test senaryoları çalıştırma, günlük kayıtlarını inceleme ve olası hata nedenlerini daraltma.
- **Talep üzerine kod üretimi:** Doğal dil taleplerinden yola çıkarak kod taslağı üretme ve ilgili dosya düzenlemelerini önerme.
- **Yeniden yapılandırma:** Eski veya verimsiz bileşenleri analiz edip daha uygun eşdeğerlerle güncelleme önerileri sunma.
- **Kod analizi ve dokümantasyon:** Kod tabanının mimarisini, veri akışlarını ve bileşen sorumluluklarını açıklayan teknik dokümanlar üretme.

Bu projenin motivasyonu, kapsamlı kod tabanı analizini mümkün olduğunca insan müdahalesi olmadan yürütebilen, izlenebilir ve ölçeklenebilir bir sistem tasarlamaktır. Nihai hedef; özellikle öğrenci ve yeni başlayan geliştiriciler için, karmaşık yazılım sistemlerini basitten zora öğreten eğitim materyallerini otomatik üretmektir.

Bu kapsamda geliştirilen örnek uygulama, iki aşamalı bir üretim hattı (*pipeline*) izler [2]: (i) yönetici ajan (*supervisor agent*) ve alt-ajanlar (*sub-agents*) aracılığıyla kod tabanından yapılandırılmış bir bilgi tabanının derlenmesi ve (ii) bu bilgi tabanının girdi olarak kullanıldığı eğitim içeriği üretimi.

Çalışmanın öne çıkan katkıları aşağıdaki başlıklar altında özetlenebilir:

- **Kararlı görev bölme yaklaşımı:** Kod tabanının dizin yapısına dayalı yalın bir kuralla alt görevlerin tanımlanması.
- **İzole çalışma alanları:** Alt-ajanların yalnızca kendi çalışma alanlarına yazabildiği, kod tabanına ise yalnızca okuma yetkisiyle eriştiği güvenli yürütüm modeli.

- **İki aşamalı üretim hattı:** Bilgi tabanı derleme ve eğitim içeriği üretimi aşamalarının ayrıştırılmasıyla kapsam ve tutarlılığın artırılması. Bu çalışmada, bilgi tabanı kullanan (Deep Agent) ve kullanmayan (Baseline) koşullar karşılaştırılmıştır.

1.2 Ön İnceleme

Bu soruna yönelik farklı yaklaşımlar bulunmaktadır. Tek ajanlı sistemler, sınırlı kapsamlı görevlerde etkili olabilir; ancak çok sayıda dosya ve bileşen içeren büyük kod tabanlarında bağlam yönetimi zorlaşmakta, uzun görevlerde odak kaybı görülebilmekte ve çıktıların tutarlılığı azalabilmektedir.

Yönetici-işçi kurgusuna dayalı çok ajanlı yaklaşımlar, görevleri parçalara ayırarak kısmi iyileştirme sağlasa da; kritik kararların ve birleştirme sorumluluğunun merkezdeki yönetici ajanda yoğunlaşması bağlam baskısını artırabilmekte ve ölçeklenebilirliği sınırlandırmaktadır.

Bu çalışma, bu dar boğazları azaltmak üzere, bağlam yükünü daha dengeli dağıtan ve ara çıktıları kalıcı bir bilgi tabanında biriktiren alt-ajan mimarisine odaklanır. Bu çerçevede geliştirilen sistem, yönetici ajan ile alt-ajanlar arasında görev bölümü yaparak kod tabanını parçalara ayırır; alt-ajanların ürettiği ara özetleri kalıcı depolarda saklar ve bu birikimi eğitim içeriği üretim aşamasına girdi olarak kullanır. Böylece hem bağlam yönetimi sadeleşir hem de üretim çıktıları denetlenebilir ara çıktılarına dayandırılır.

Literatürde ajan mimarilerinde iki temel paradigma öne çıkmaktadır. **ReAct** [1], LLM'lerin düşünce-eylem-gözlem döngüsüyle dış araçları kullanmasını sağlar. **ReWOO** [3] ise planlama ve yürütme aşamalarını ayırarak token verimliliğini artırmaktadır.

Çoklu ajan sistemlerinde ise **Mixture of Agents (MoA)** [4] ve **CAMEL** [5] yaklaşımları, farklı uzmanlıklara sahip ajanların işbirliğiyle tekil modellerden üstün sonuçlar elde edilebileceğini göstermiştir. Bu çalışma, **ReWOO**'nun modüler yapısını benimseyerek geniş kod tabanlarını bağlam taşması olmadan analiz etmeyi hedeflemektedir.

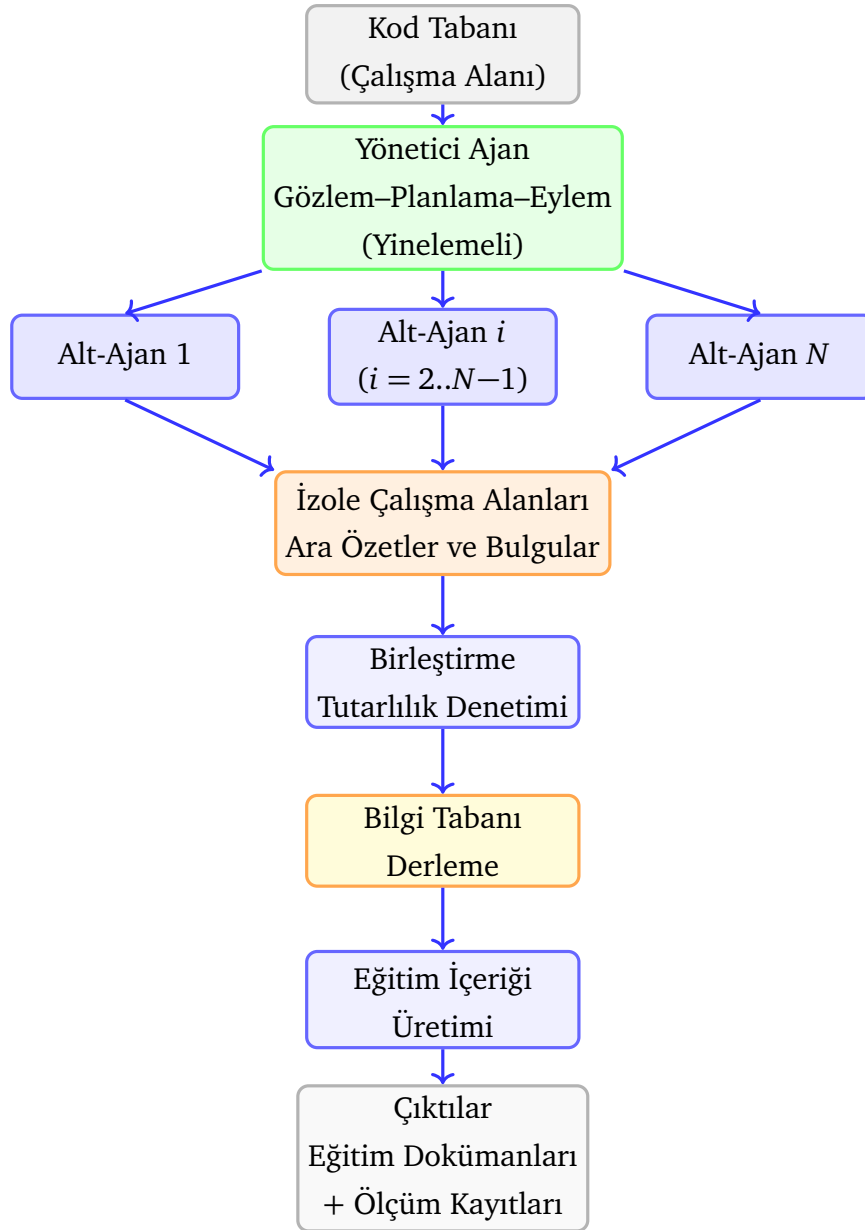
2 SİSTEM ANALİZİ VE FİZİBİLİTE

2.1 Sistem Analizi

Bu bölümde, çok ajanlı yazılım sisteminin kapsamı ve hedefleri ele alınmaktadır. Şekil 2.1’de sunulan mimaride temel amaç, hedef kod tabanının hiyerarşik bir ajan yapısı aracılığıyla çözümlenmesi, elde edilen bulguların yapılandırılmış bir **bilgi tabanı** biçiminde derlenmesi ve bu birikimin pedagojik değer taşıyan **eğitim içeriklerine** dönüştürülmesidir. Söz konusu yaklaşım, yalnızca dokümantasyon üretiminin ötesine geçerek; kaynak koddan türetilen bilginin sistematik biçimde yapılandırılmasına, izlenebilir şekilde birleştirilmesine ve öğretici bir kurgu içinde sunulmasına odaklanmaktadır.

Deep Agent mimarisi, planlama ve yürütmenin ayrıştırılması yaklaşımından esinlenerek iki aşamalı bir alt-ajan kurgusu üzerine inşa edilmiştir [3]. Sistem, Hugging Face tarafından geliştirilen **smolagents** çatısı üzerine kurulmuştur [6]. Bu çatı, büyük dil modellerinin araç çağırısı yapabilmesini sağlayan hafif bir ajan altyapısı sunar; ajanların dinamik olarak oluşturulmasını, araç takımlarının modüler biçimde tanımlanmasını ve çok adımlı görevlerin yürütülmesini destekler. Yönetici ajan; keşif, planlama ve eşgüdüm süreçlerini yürütmekte olup, bağlam yükünü hafifletmek amacıyla görevleri alt-ajanalara devretmektedir. Alt-ajanlar ise izole çalışma alanlarında faaliyet göstererek analizlerini gerçekleştirmekte, bulgularını ara depolara kaydetmekte ve yönetici ajana raporlamaktadır. Yönetici ajan, bu çıktıları sentezleyerek bütünlük bilgisi tabanını oluşturmakta; ardından eğitim içeriği üretim sürecini başlatmaktadır.

Görev bölümlendirme stratejisi açısından, tipik olarak kaynak dizin altındaki birinci düzey klasörler ayrı alt-ajanalara atanmaktadır; ancak görevler farklı granülerlikte de bölünebilir. Alt-ajan sayısı N sabit değildir; yönetici ajan, kod tabanı ağaç yapısı ve oluşturduğu görev listesine bağlı olarak çalışma anında N adet alt-ajan çalışma alanı başlatır. Buna ek olarak, yönetici ajanın planlama parametreleri ve dinamik görev listesi üzerinden çalışma anında görev dağılımını güncelleyebilmesi mümkündür. Alt-ajanların kod tabanına yalnızca okuma, kendi çalışma alanlarına ise yazma yetkisine sahip olması hem güvenliği güçlendirmekte hem de çıktıların izlenebilirliğini artırmaktadır.



Şekil 2.1 Deep Agent mimarisinde yönetici ajan, alt-ajanlar ve bilgi tabanı üzerinden eğitim içeriği üretim akışının üst düzey görünümü.

Şekil 2.1’de sunulan mimari temsili bir görünüm olup, alt-ajan sayısı örnek olarak $N = 3$ şeklinde gösterilmiştir. Gerçek yürütümde alt-ajan sayısı (N), yönetici ajan tarafından kod tabanının yapısına ve oluşturulan görev listesine bağlı olarak dinamik biçimde belirlenmektedir.

Başarı Kriterleri ve Metrikler

Sistemin başarımı, formel doğruluk formülleri yerine; sistem bileşenleri tarafından derlenen işletim verileri ve süreç yürütümlerine ilişkin izler üzerinden değerlendirilmektedir.

- **Operasyonel Verimlilik:** Sistemin bir projeyi analiz etme maliyeti üç temel göstergeyle izlenir:
 - **Token Tüketimi (*token consumption*):** Toplam girdi/çıkıtı token sayısı (maliyet analizi için).
 - **Süre:** Bilgi tabanı ve eğitim içeriği üretimi aşamalarının tamamlanma süreleri.
 - **Araç Çağrılarını (*tool calls*):** Ajanların dosya okuma ve yazma işlemlerinin sıklığı.
- **Süreç Dayanıklılığı:**
 - **Bağlam Taşması (*context overflow*):** Büyük dil modelinin bağlam sınırını aşma olaylarının sayısı.
 - **Hata Sayısı:** Yürütüm sırasında oluşan ve günlüğe kaydedilen hata olaylarının sayısı.

2.2 Gereksinim Analizi

Gereksinimler üç temel katmanda sınıflandırılmıştır: kullanıcı rolleri, sistem modülleri ve işlevsel akış. Önerilen sistemde temel kullanıcı profili *geliştiricidir*. Geliştirici, ajan süreçlerini başlatıp izleyebilir; yürütüm sonunda üretilen bilgi tabanı ve eğitim çıktıları aracılığıyla kod tabanını öğrenir ve çıktıların bütünlüğünü doğrular.

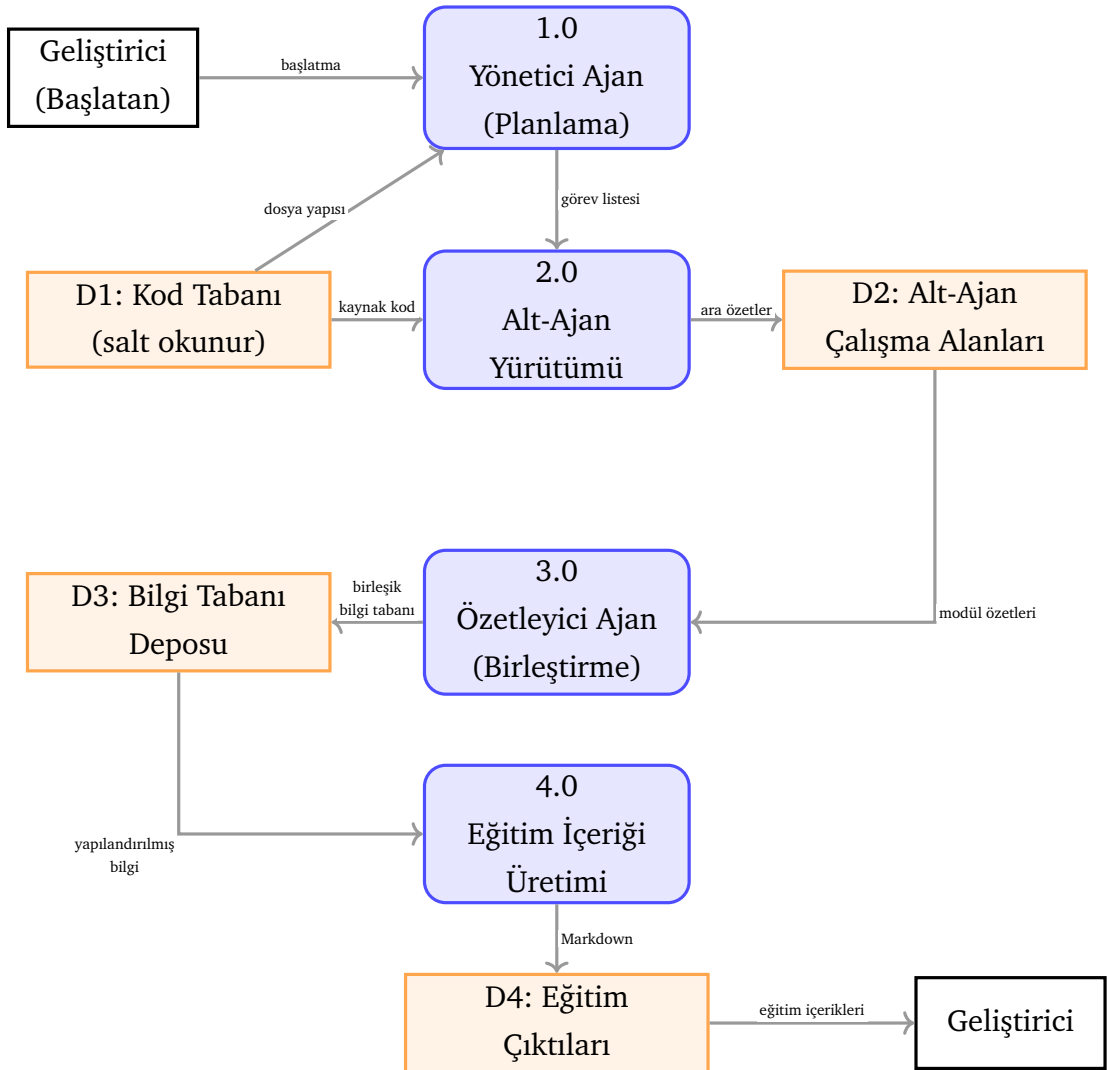
Sistem modülleri dört ana bileşenden teşekkül etmektedir. **Yönetici ajan**, keşif, planlama, görev dağıtımı ve alt-ajanlardan gelen çıktıların birleştirilmesi görevlerini üstlenen merkezî unsur olarak konumlandırılmıştır. **Alt-ajanlar**, deterministik veya koşula bağlı biçimde üretilen, belirli dizinleri ya da görev kümelerini çözümleyen ve izole çalışma alanlarında faaliyet gösteren bileşenlerdir. **Bilgi tabanı deposu**, tüm alt-ajan özetlerinin toplandığı yapısal bir düzen sunarken; **eğitim içeriği üretim bileşeni** bu birikimi kullanarak pedagojik değer taşıyan materyalleri bağımsız bir çıktı alanı altında üretmektedir.

Sistemin işlevsel akışı ardışık beş aşamada gerçekleştirilmektedir. Birinci aşamada yönetici ajan, kod tabanının dosya yapısını çıkarmakta, kritik bileşenleri çözümlemekte ve deterministik ya da dinamik olarak görev listesini oluşturmaktadır. İkinci aşamada bu görevler ilgili alt-ajanalara tevdi edilmektedir. Üçüncü aşamada alt-ajanlar, sorumluluk alanlarındaki kod parçalarını inceleyerek özetlerini kendi çalışma dizinlerine kaydetmektedir. Dördüncü aşamada yönetici ajan, alt çıktıları sentezleyerek bütünlük bilgisi tabanını oluşturmakta ve kalite kontrolünü gerçekleştirmektedir. Son aşamada

eğitim içeriği üretim bileşeni, bu bilgi tabanını geliştiricilerin kolayca anlayabileceği pedagojik bir formata dönüştürmektedir.

İşlevsel olmayan gereksinimler kapsamında, sistemin güvenlik, izlenebilirlik, genişletilebilirlik ve performans özellikleri tanımlanmaktadır. Sistem, özellikle yetkisiz dizin geçişi zafiyetlerine karşı güvenli olacak şekilde tasarlanmalıdır; tüm API anahtarları gizli tutulmalı ve kaynak koda gömülmemelidir. Bunun yanı sıra, ajanların karar verme süreçleri—özellikle görev devri ve dosya işlemleri—izleme bileşeni üzerinden ayrıntılı biçimde kayıt altına alınmalıdır. Mimarının yeni araçların veya ajan türlerinin eklenmesine açık olması gerekmektedir. Performans açısından, ilk sürümde karmaşıklığı sınırlamak amacıyla alt-ajanların paralel değil sıralı çalışması tercih edilmiştir.

Şekil 2.2’de sistemin veri akış diyagramı sunulmaktadır. Bu diyagram, geliştiricinin süreci başlatmasından üretilen eğitim çıktılarının geliştiriciye sunulmasına kadar olan veri akışını ve ara işlem adımlarını göstermektedir.



Şekil 2.2 Sistem veri akış diyagramı.

2.3 Bilgi Tabanı Sistemi

Deep Agent mimarisinin temelini oluşturan bilgi tabanı (Knowledge Base - KB), eğitim içeriği üretiminden önce kod tabanının yapılandırılmış bir analizini sağlar. Bu bölümde KB'nin amacı, üretim süreci, dosya yapısı ve eğitim hattıyla entegrasyonu açıklanmaktadır.

2.3.1 Amaç ve Motivasyon

Büyük dil modellerinin bağlam penceresi kısıtları, kapsamlı bir kod tabanının tek seferde işlenmesini zorlaştırmaktadır. Bilgi tabanı bu soruna aşağıdaki çözümleri sunar:

- **Bağlam Sıkıştırma:** Her modül için yoğunlaştırılmış özet üreterek token tüketimini azaltır.
- **Yapısal Rehberlik:** Eğitim üretim ajanına “nereye bakacağını” gösteren bir navigasyon haritası sağlar.
- **Doğruluk Artışı:** Önceden doğrulanmış API imzaları ve dosya yollarıyla halüsinasyon riskini düşürür.
- **Yeniden Kullanılabilirlik:** Aynı KB, farklı eğitim serilerinde veya farklı modeller tarafından tekrar kullanılabilir.

2.3.2 Üretim Süreci

Bilgi tabanı üretimi, yönetici ajan koordinasyonunda dört aşamadan oluşur:

1. **Keşif ve Planlama:** Yönetici ajan, *kod ağacı çıkarma aracı* ile proje yapısını tarar ve kaynak dosyaları işlevsel gruplara ayırır. Ardından bir görev planı oluşturarak hangi dosyaların hangi alt-ajan tarafından analiz edileceğini belirler. Varsayılan yapılandırmada en fazla 6 görev ve görev başına en fazla 4 dosya ile sınırlandırılmıştır; bu değerler sistemin **yapılandırma ayarları** üzerinden değiştirilebilir.
2. **Sıralı Analiz:** Her görev için bir *analizci alt-ajan* başlatılır. API limit koruması amacıyla ajanlar sıralı olarak çalıştırılır. Atanan dosyaların varlığı önceden doğrulanır ve görev tamamlandığında plan otomatik güncellenir. Kurtarma noktası mekanizması sayesinde çökme sonrası yarım kalan işlemler devam ettirilebilir.
3. **Birleştirme:** Tüm alt-ajanlar tamamlandığında *toplayıcı* devreye girer. Bu bileşen, dağınık çalışma alanlarından özet dosyalarını otomatik keşfeder ve tek bir bilgi tabanı dizininde birleştirir.

4. **Yönetici Özeti:** Son adımda *özetleyici ajan*, modül raporlarını okuyarak projenin üst düzey haritasını oluşturur. Bu özet, eğitim ajanına “hangi konuyu nerede bulabilirim” sorusuna yanıt verir.

2.3.3 Dosya Yapısı ve İçerik

Ortalama bir proje için üretilen bilgi tabanı, Tablo 2.1’te özetlenen bileşenlerden oluşmaktadır.

Tablo 2.1 Bilgi tabanı bileşenleri ve tipik boyutları.

Kategori	Açıklama	Tipik boyut (kB)
Görev planı özeti	Analiz görevlerinin kontrol listesi	0.5–1
Yönetici özeti	Projenin üst düzey gezinme haritası	4–5
Modül raporları (×4–6)	Modül bazlı teknik analiz	5–9 (adet başı)
Toplam	–	30–50

Her modül raporu; modülün üst düzey amacını, temel bileşenlerin işlevsel özetini, dışa açık arayüz tanımlarını, diğer modüllerle etkileşim noktalarını ve kullanım senaryolarını içerir. Yönetici özeti ise projenin genel amacını, konu–bileşen eşleme özetini ve kritik bileşenlerin kısa tanımlarını barındırır.

2.3.4 Eğitim Hattıyla Entegrasyon

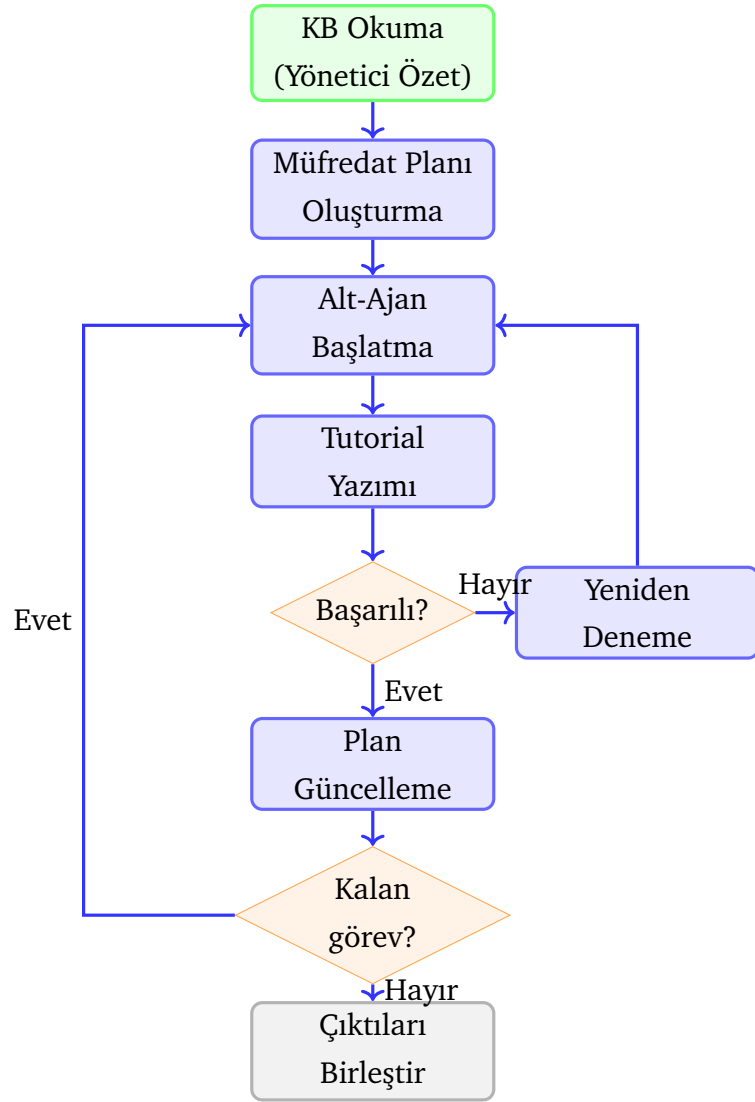
Eğitim içeriği üreten ajan, bilgi tabanını iki adımlı bir süreçle kullanır:

1. **Keşif:** Önce *bilgi tabanı kataloglayıcı* ile mevcut özet dosyalarının listesini alır; ardından *özet okuyucu* ile ilgili modül raporlarını inceler.
2. **Doğrulama:** Bilgi tabanından edindiği yapısal bulguları, kaynak kod inceleme aracıyla özgün içerikten çapraz kontrol eder.

Bu iki aşamalı yaklaşım, bilgi tabanının rehberlik sağlarken özgün yazılım içeriğinin tek doğruluk kaynağı olarak kalmasını garanti eder.

2.3.5 Eğitim İçeriği Üretim Süreci

Eğitim içeriği üretimi, bilgi tabanı oluşturma aşamasından sonra devreye giren ikinci ana fazdır. Bu aşamada yönetici ajan “Eğitim Serisi Direktörü” rolünü üstlenerek, hedef kod tabanı için altı adede kadar tutarlı öğretici doküman üretilmesini koordine eder. Şekil 2.3’te bu sürecin akış diyagramı sunulmaktadır.



Şekil 2.3 Eğitim içeriği üretim süreci akış diyagramı.

Üretim süreci üç fazdan oluşmaktadır. İlk fazda (**Keşif ve Planlama**) yönetici ajan, bilgi tabanından yönetici özeti ve modül raporlarını okuyarak projenin ana amacını ve yapısını kavrar. Ardından hangi konuların hangi sırayla işleneceğini belirleyen bir müfredat planı oluşturur. İkinci fazda (**Sıralı Yürütüm**) her tutorial için ayrı bir alt-ajan başlatılır. Alt-ajanlar, API hız sınırı koruması amacıyla sıralı olarak çalıştırılır; bir alt-ajan tamamlanmadan diğeri başlatılmaz. Tamamlanan görevler kontrol listesinde işaretlenir. Üçüncü fazda (**Doğrulama ve Yayın**) eksik veya hatalı çıktılar tespit edilir, yeniden deneme mekanizması devreye girer ve nihai dosyalar birleştirilir.

Eğitim Araç Takımı: Eğitim alt-ajanları, bilgi tabanı ve kaynak kod ile etkileşim için özelleştirilmiş araçlar kullanır:

- **Bilgi tabanı kataloglama:** Mevcut KB dosyalarının listesini döndürür.

- **KB dosya okuma:** Belirli bir modül raporunu veya yönetici özetini okur.
- **Kaynak kod okuma:** Orijinal dosyaları satır aralıklarıyla okuyarak bağlam tüketimini kontrol eder.
- **Çalışma alanına yazma:** Üretilen tutorial'ı Markdown formatında kaydeder.

Yeniden Deneme Mekanizması: Bir alt-ajan başarısız olduğunda veya çıktı kalite kontrolünden geçemediğinde, sistem yapılandırılmış geri bildirimle yeniden deneme başlatır. Biçimlendirme hataları için hedefli düzeltme; içerik eksiklikleri için tam yeniden üretim uygulanır. Maksimum deneme sayısına ulaşıldığında görev atlanır ve günlüğe kaydedilir.

Pedagojik açıdan sistem belirli ilkelere bağlı kalır: ilk tutorial projenin ana bileşenini tanıtır (“Hello World” yaklaşımı); içerikler kolaydan zora doğru yapılandırılır; her tutorial bir öncekinin üzerine inşa edilir (kümülatif öğrenme); ve temel API, gelişmiş seçenekler, komut satırı kullanımı ile hata yönetimi gibi kritik konular kapsanır.

Deep Agent ve Baseline koşulları arasındaki temel fark, bilgi tabanı erişimi ve istem tasarımıdır. Her iki koşulda da aynı araç takımı (dosya okuma, yazma, izin listeleme) kullanılmaktadır. Ancak Deep Agent koşulunda bilgi tabanı araçları aktif olup, istemler bu kaynaktan yararlanmaya yönelik özelleştirilmiş yönergeler içerir. Baseline koşulunda ise bilgi tabanı araçları devre dışıdır ve ajan yalnızca doğrudan kaynak kod inceleme araçlarıyla çalışır. Bu tasarım, bilgi tabanının eğitim içeriği kalitesi üzerindeki marjinal katkısının izole biçimde ölçülmesine olanak tanır.

2.3.6 Ampirik Gözlemler

7 Ocak 2026 tarihli *deepagents* [7] yürütümünde bilgi tabanı aşaması şu metrikleri üretmiştir: 724.9 saniye süre, 42 araç çağrısı ve 6 alt-ajan. Çıktı olarak 8 dosya (~47 KB) oluşturulmuştur. Aynı projenin Baseline koşulunda bilgi tabanı aşaması bulunmadığından, toplam yürütüm süresi 338 saniyeye düşmüş; ancak kalite değerlendirmelerinde belirgin fark gözlemlenmiştir.

Bu veriler, KB üretiminin toplam Deep Agent yürütüm süresinin yaklaşık %58'ini oluşturduğunu; buna karşın eğitim fazındaki hedefli dosya erişimi ve azaltılmış halüsinasyon oranıyla bu yatırımın kendini amorti ettiğini göstermektedir (bkz. Bölüm 4.2).

2.4 Fizibilite

2.4.1 Teknik Fizibilite

2.4.1.1 İş Yükü Profili ve Ölçülebilir Sınırlar

Önerilen sistem, **girdi-çıkı (I/O)** ve ağ ağırlıklı bir iş yükü profiline sahiptir. Ana kaynak tüketimi; çok sayıda dosya okuma işlemi, LLM servis çağrıları ve ara çıktıların diske yazılması biçiminde gerçekleşmektedir. Bu profil, yüksek işlemci veya grafik işlem birimi gereksinimine yol açmamakta; bunun yerine ağ gecikmesi ve **API hız sınırı (rate limit)** darboğaz noktaları olarak öne çıkmaktadır.

Ölçülebilir sınırlar:

- **Proje büyüklüğü:** Deneylerde beş açık kaynak proje (agent-lightning, deepagents, instructor, RAG-Anything, toon) başarıyla işlenmiştir. Bu projeler farklı boyut ve mimari özelliklere sahip olup, daha büyük projeler için görev bölümlleme stratejisinin özelleştirilmesi gerekebilir.
- **Bağlam sınırı:** LLM bağlam penceresi aşımı (context overflow) riski, alt-ajan mimarisi ve özet temsilleriyle yönetilmektedir. Deneylerde bu tür bir hata gözlenmemiştir.
- **Süre beklentisi:** Bölüm 4.2'deki ölçümlere göre, Deep Agent toplam süresi yaklaşık 15–24 dakika, Baseline ise 5–7 dakika aralığında tamamlanmıştır.

2.4.1.2 Donanım Gereksinimleri

Model eğitimi hedeflenmediğinden yüksek donanım yatırımı gereksinimi bulunmamaktadır. Alt-ajanların sıralı yürütülmesi, bellek ve CPU baskısını kontrol altında tutmaktadır. Tablo 2.2, minimum ve önerilen yapılandırmaları özetlemektedir.

Tablo 2.2 Donanım gereksinimleri.

Bileşen	Minimum	Önerilen
CPU	2 çekirdek	4+ çekirdek
RAM	4 GB	8+ GB
Disk	10 GB	20+ GB (büyük projeler için)
Ağ	Stabil internet	Düşük gecikmeli bağlantı
GPU	Gerekli değil	Yerel model için önerilir

2.4.1.3 Yazılım ve Çalıştırma Ortamı

Teknolojik bileşen kümesi:

- **İşletim Sistemi:** Windows, macOS veya Linux.
- **Uygulama bileşenleri:** Sunucu tarafında bir uygulama katmanı; istemci tarafında ise web tabanlı bir kullanıcı arayüzü kullanılmaktadır.
- **Bileşenler:** Ajan eşgüdümü, model erişimi (geçit katmanı), web arayüzü ve izleme bileşenleri açık kaynak araçlarla sağlanır.

Bağımlılık riski ve azaltım: Model sağlayıcı kesintisi veya arayüz değişikliği durumunda, model erişim ara katmanı üzerinden alternatif bir modele geçiş yapılabilir. Bu tasarım, sağlayıcı bağımlılığını azaltmaktadır.

Model Seçimi ve Sağlayıcı Yapılandırması: Bu çalışmada geliştirme ve test sürecinde bulut tabanlı bir büyük dil modeli hizmetinden yararlanılmıştır. Sistem, model erişimini ara katman üzerinden yapılandırılabilir kıldığından, kullanıcılar isterlerse farklı bir sağlayıcıya geçebilir veya yerel/açık kaynak bir modeli kullanabilir. Bu durumda, yürütüm ortamına göre kimlik doğrulama bilgileri, hız sınırı ve benzeri erişim parametrelerinin uygun biçimde yapılandırılması gerekmektedir.

2.4.2 Yasal Fizibilite

Sistemde kullanılan kütüphaneler MIT veya Apache 2.0 benzeri açık kaynak lisansları kapsamında dağıtılmaktadır.

API ve veri gizliliği:

- Kod tabanı içeriği LLM API'ye gönderildiğinden, yalnızca **açık kaynak veya kullanım yetkisi bulunan** projeler analiz edilmelidir.
- Gizli anahtarlar ve hassas veriler, ve ortam değişkenleri ile maskelenerek korunmaktadır.
- Ticari/kapalı kaynak projeler için yerel modellerin kullanımı önerilmektedir.

Çıktı lisansı: Üretilen eğitim içerikleri, kaynak kod tabanının lisansına tabi olup; dağıtım öncesinde ilgili lisans koşullarının gözden geçirilmesi gerekmektedir.

2.4.3 Ekonomik Fizibilite

2.4.3.1 Maliyet Modeli

Toplam maliyet üç ana kalemden oluşmaktadır:

$$\begin{aligned}\text{Toplam maliyet} &= \text{Model hizmeti kullanım gideri} \\ &+ \text{Mühendislik emeği} \\ &+ \text{Altyapı ve otomasyon giderleri}\end{aligned}$$

Geliştirme Sürecinde Gerçekleşen Maliyet: Geliştirme sürecinde gerçekleştirilen deneysel koşullar ve karşılaştırmalı testler kapsamında, bulut tabanlı model hizmetine erişim için dönemsel ücretli kullanım gerekebilmiştir. Toplam gider; seçilen hizmetin fiyatlandırmasına, yürütüm sayısına, üretilen çıktı uzunluğuna ve yeniden deneme mekanizmalarının devreye girme sıklığına bağlı olarak değişkenlik gösterebilmektedir.

Token Tüketimi: Token tüketimi, model hizmeti maliyetinin doğrudan belirleyicisidir; ancak pratikte fiyatlandırma detayları ve ölçüm tutarlılığı nedeniyle proje bağlamında tek başına güvenilir bir maliyet özeti sunmak her zaman mümkün olmayabilir. Bu nedenle çalışma kapsamında ekonomik değerlendirme; yürütüm sıklığı, çıktı beklentisi ve zaman kazanımı gibi operasyonel değişkenlerle birlikte ele alınmıştır.

2.4.3.2 Kullanım Senaryoları

- **Düşük sıklık** (ayda 1–2 kez): Yeni projeye uyum veya dokümantasyon güncelleme; maliyet ihmal edilebilir düzeyde.
- **Orta sıklık** (haftada 1–2 kez): Sürekli entegrasyon döngüsünde periyodik yeniden üretim.
- **Yüksek sıklık** (günlük): Büyük ekiplerde sürekli güncel dokümantasyon; model seçimi ve önbellekleme ile maliyet optimizasyonu gerekebilir.

2.4.3.3 Katma Değer

Sistemin sunduğu katma değer, doğrudan parasal kazançtan ziyade zaman tasarrufu ve verimlilik artışı üzerinden tezahür etmektedir:

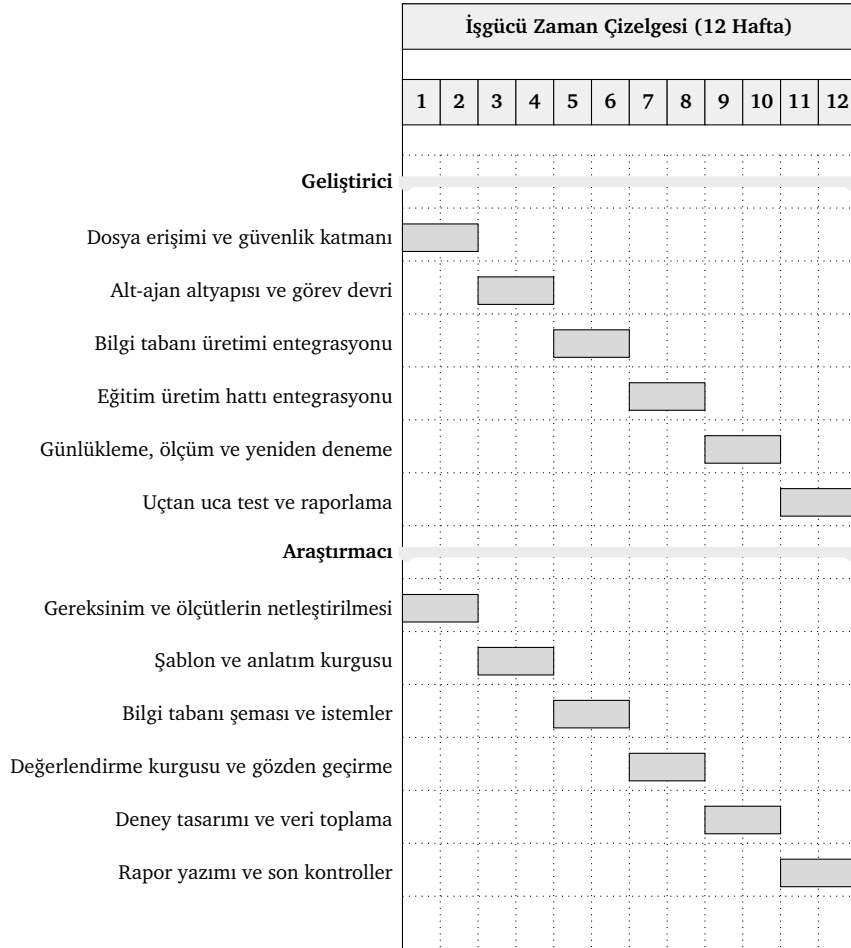
- Dokümantasyon ve proje öğrenme süreçlerinde belirgin zaman tasarrufu.
- Yeni geliştiricilerin projeye uyum süresinin kısılması.

- Yazılım sistemi değıştikçe ajanların yeniden çalıştırılmasıyla daha güncel dokümantasyon çıktıları.

2.4.4 İşgücü ve Zaman Fizibilitesi

Ekip: 2 kişi (yazılım/araştırma odaklı).

Genel Yaklaşım: İlk sürüm için 8–12 hafta aralığında bir geliştirme süresi öngörölmüşür. Zaman planı, rol bazlı görev dağılımıyla Şekil 2.4’de özetlenmiştir.



Şekil 2.4 İşgücü fizibilitesi: rol bazlı zaman çizelgesi.

3 SİSTEM TASARIMI

Bu bölümde, geliştirilen çok ajanlı yazılım sisteminin tasarımı ayrıntılandırılmaktadır. Tasarım; kullanıcı etkileşimi, veri yönetimi ve yazılım mimarisi olmak üzere üç eksenle ele alınmıştır. Amaç, karmaşık yazılım projelerini yarı-özerk biçimde analiz edebilen ve pedagojik içerik üretebilen, modüler ve güvenli bir yapı ortaya koymaktır.

3.1 Arayüz Tasarımı

Sistem, kullanım kolaylığı ile operasyonel esnekliği birlikte sağlayabilmek amacıyla ikili bir arayüz yaklaşımı benimsemiştir: süreçleri başlatmak ve yönetmek için **komut satırı arayüzü** (*CLI*), üretilen çıktıları incelemek ve düzenlemek için **web tabanlı kontrol paneli**.

3.1.1 Komut Satırı Arayüzü

Analiz süreçlerinin başlatılması ve yönetilmesi, geliştiricilerin alışkın olduğu komut satırı ortamı üzerinden gerçekleştirilir. Kullanıcılar, tek bir komut ile hedef projeyi sisteme tanıtır ve analiz sürecini başlatır. Komut satırı arayüzü, arka planda çalışan yapay zekâ bileşenlerinin durumunu, kaynak tüketimini ve ilerleme yüzdesini anlık olarak ekrana yansıtır. Bu yalın tasarım, sistemin sürekli entegrasyon ve sürekli teslim (*CI/CD*) süreçlerine kolayca entegre edilmesine olanak tanır.

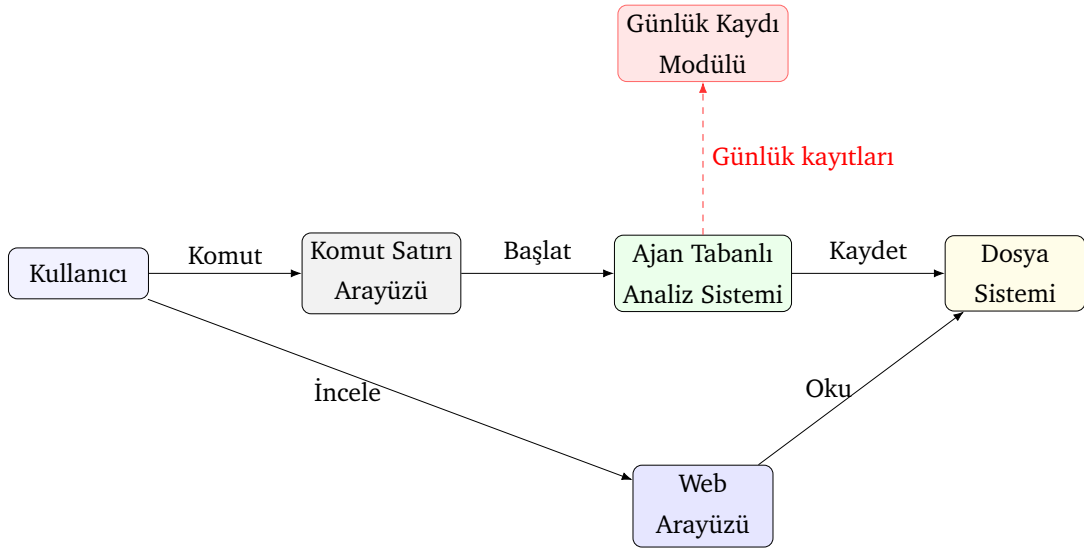
3.1.2 Web Tabanlı Kontrol Paneli

Analiz tamamlandıktan sonra üretilen bilgi tabanı ve eğitim materyallerinin son kullanıcıya sunulması için web tabanlı bir arayüz sağlanmıştır. Arayüzün temel işlevleri şunlardır:

- **Proje Kütüphanesi:** Analiz edilen farklı kod tabanlarının kart tabanlı görünümde listelenmesi ve yönetilmesi.

- **Etkileşimli İçerik Görüntüleyici:** Ajanlar tarafından üretilen Markdown biçimindeki ders içeriklerinin, söz dizimi renklendirme ve diyagram desteğiyle okunabilir hâle getirilmesi.
- **Hiyerarşik Gezinme:** Oluşturulan müfredat içerikleri arasında kademeli geçiş imkânı.

Şekil 3.1’te sistemin operasyonel ve analitik arayüzleri arasındaki veri akış diyagramı sunulmuştur.



Şekil 3.1 Kullanıcı etkileşimi ve veri akış diyagramı.

3.2 Veri Yapısı Tasarımı

Proje mimarisinde, karmaşık veritabanı yönetim sistemleri yerine taşınabilirliği ve şeffaflığı önceleyen dosya sistemi tabanlı (*file-based*) bir veri yaklaşımı tercih edilmiştir. Bu tasarım kararı, üretilen dokümantasyonun sürüm kontrol (*version control*) süreçleriyle uyumlu biçimde tutulabilmesini ve projenin kendisiyle birlikte taşınabilmesini sağlar.

3.2.1 Yapılandırma Yönetimi

Sistemin davranış kalıpları ve parametreleri, merkezi yapılandırma dosyaları (*configuration files*) üzerinden yönetilir. Model seçimleri, API erişim anahtarları ve genel sistem ayarları tek bir noktadan kontrol edilirken, web arayüzünde görüntülenecek üstveri (*metadata*) bilgileri ayrı bir yapılandırma katmanında tutulur.

3.2.2 İzole Çalışma Alanları (Kısıtlı Dosya Sistemi)

Veri güvenliğini sağlamak ve projeler arası çakışmaları önlemek amacıyla, analiz sürecinin yalnızca izinli konumlar üzerinde işlem yapabildiği **izole çalışma alanı** (*sandbox*) yaklaşımı uygulanmıştır. Her analiz süreci, kendine tahsis edilen üç katmanlı bir çalışma alanı yapısı içinde çalışır:

1. **Kaynak Katmanı (Okuma):** Ajanların sadece okuma iznine sahip olduğu, orijinal kod tabanının bulunduğu güvenli alan.
2. **Geçici Katman (Ara Depolama):** Analiz sürecinde oluşturulan ara özetlerin ve notların tutulduğu çalışma alanı.
3. **Çıktı Katmanı (Yazma):** Nihai eğitim materyallerinin ve raporların derlendiği sonuç klasörü.

Bu yapı sayesinde, yapay zekâ ajanlarının proje dosyalarına kontrolsüz erişimi engellenmiş ve veri bütünlüğü garanti altına alınmıştır.

3.3 Nesneye Yönelik Tasarım ve Ajan Mimarisi

Sistemin yazılım mimarisi, statik sınıf hiyerarşileri yerine esnekliği artıran **bileşim temelli** (*composition-based*) ve **dinamik** bir tasarım modeline dayanır. Ajanlar, sabit kod blokları olarak değil; çalışma zamanında (*runtime*) ihtiyaca göre bir araya gelen yetenek kümeleri olarak kurgulanmıştır.

3.3.1 Dinamik Ajan Yapısı

Platform içerisindeki her bir ajan (yönetici ajan, alt-ajan ve eğitim içeriği üretim bileşeni), üç temel bileşenin dinamik birleşimiyle oluşturulur:

- **Kimlik ve Rol Tanımı (İstem/*prompt*):** Ajanın amacını, kurallarını ve davranış biçimini belirleyen doğal dil talimatlarıdır.
- **Yetenek Seti (Araç Takımı/*toolkit*):** Ajanın dosya okuma/yazma ve komut yürütme gibi teknik becerilerini sağlayan modüler fonksiyon kümeleridir.
- **Büyük Dil Modeli:** Görevin karmaşıklığına göre seçilen ve ajanın çıkarım/üretim yeteneğini sağlayan işlem birimidir.

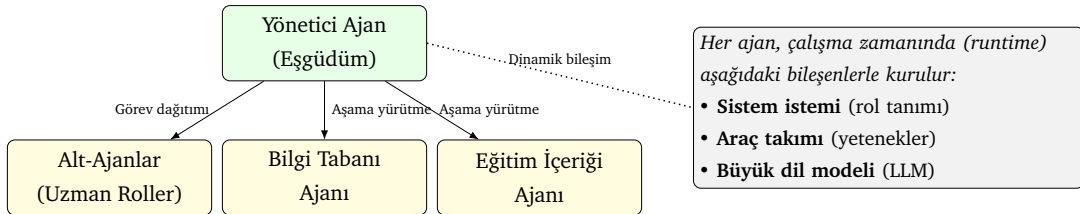
Bu modüler yapı, sisteme yeni yeteneklerin eklenmesini veya ajan davranışlarının değiştirilmesini, kaynak kodda köklü değişiklikler yapmadan mümkün kılar.

3.3.2 Eşgüdüm Katmanı

Ajanların koordinasyonu ve görev dağılımı, üst seviye yönetici bileşen tarafından sağlanır [4]. Bu eşgüdüm katmanı (*orchestration layer*), analiz sürecini mantıksal aşamalara (bilgi toplama, planlama, içerik üretimi) böler ve her aşama için gerekli ajanları sıralı ve denetlenebilir bir biçimde devreye sokar.

Yürütüm Modeli (Sıralı Alt-Ajan Zamanlama): Mevcut uygulamada alt-ajanlar **sıralı** (ardışık) olarak çalıştırılmaktadır; yani bir alt-ajan görevi tamamlamadan bir sonraki başlatılmaz. Yönetici ajan, görev listesini FIFO (First In, First Out - ilk giren ilk çıkar) mantığında kuyruklar ve her alt-ajanı teker teker başlatarak çıktısını bekler. LLM API hız sınırı (*rate limit*) aşıldığında sistem, artan bekleme sürelili yeniden deneme (*exponential backoff*) uygular; ancak görev düzeyinde otomatik yeniden deneme mevcut sürümde desteklenmemektedir. Başarısız görevler günlüğe kaydedilir ve bir sonraki göreve geçilir. Bu yaklaşım, paralel yürütüme kıyasla daha uzun toplam süre anlamına gelse de; kaynak tüketimini kontrol altında tutar, hata ayıklamayı kolaylaştırır ve çıktıların deterministik sırada üretilmesini sağlar.

Şekil 3.2’de eşgüdüm katmanında yer alan temel ajan rolleri ile her bir ajanın çalışma zamanında *istem + araç takımı + büyük dil modeli* bileşenlerinden oluşturulması ilkesi özetlenmiştir.



Şekil 3.2 Eşgüdüm katmanında ajan rolleri ve dinamik bileşim ilkesi. Alt-ajan sayısı (N) çalışma anında yönetici ajan tarafından dinamik olarak belirlenir; şekil temsili olarak tek düğüm altında gösterilmiştir.

Bu mimari tercih, sistemin bakım maliyetini düşürmekte ve farklı teknolojilere uyarlabilmesini kolaylaştırmaktadır.

3.3.3 Model Geçit Katmanı (LLM Gateway)

Sistem, büyük dil modeli çağrılarını merkezi bir geçit katmanı üzerinden yönetmektedir. Bu katman aşağıdaki işlevleri sağlar:

- **Rol bazlı model seçimi:** Yönetici ajan, alt-ajanlar ve eğitim üretim bileşeni için farklı model kimlikleri yapılandırılabilir.

- **Merkezi kimlik doğrulama:** API anahtarları ve proje bilgileri tek noktadan yönetilir.
- **Zorunlu bekleme süresi:** Ardışık istekler arasında minimum bekleme süresi uygulanarak hız sınırı aşımı riski azaltılır.
- **Artan bekleme (exponential backoff):** HTTP 429 veya “rate limit” hatalarında sistem, artan bekleme süreleriyle yeniden deneme yapar.
- **Token kullanım kaydı:** API yanıtlarındaki kullanım bilgileri geri çağırma (callback) mekanizmasıyla metrik sistemine aktarılır.

3.3.4 Araç Takımları

Ajanların dosya sistemi ve kod tabanıyla etkileşimi, kısıtlı erişim modeline sahip araç takımları aracılığıyla sağlanır. Temel araçlar şunlardır:

- **Sayfalı dosya okuma/yazma:** Büyük dosyalar belirli satır aralıklarıyla okunarak bağlam tüketimi kontrol altında tutulur.
- **Dizin listeleme:** Hedef klasördeki dosya ve alt dizinler yapılandırılmış biçimde listelenir.
- **Mermaid ağaç oluşturma:** Proje dizin yapısı, görselleştirmeye uygun Mermaid diyagramı olarak üretilir.
- **AST iskelet okuma:** Python dosyalarının soyut sözdizim ağacından sınıf ve fonksiyon imzaları çıkarılır.

Güvenlik kısıtları: Tüm dosya işlemleri, tanımlanan kök dizin dışına çıkamaz (*root confinement*). Ayrıca belirli uzantılar (örn. çalıştırılabilir dosyalar) engelleme listesine alınmıştır.

3.3.5 Bilgi Tabanı Şeması

Bilgi tabanı oluşturma fazında üretilen çıktılar, kaynak kod tabanının dizin hiyerarşisini yansıtan Markdown özet dosyaları biçiminde çalışma alanına kaydedilmektedir. Her alt-ajan, kendisine atanan modül veya bileşen için yapılandırılmış bir özet üretir; bu özetler daha sonra yönetici ajan tarafından birleştirilerek tutarlı bir bilgi tabanı oluşturulur.

3.3.6 Gözlemlenebilirlik ve Ölçüm

Sistem, yürütüm sırasında çeşitli metrikleri otomatik olarak kaydeder:

- **Çalışma süresi:** Her faz (bilgi tabanı, eğitim üretimi) için ayrı ayrı ölçülür.
- **Araç etkileşimi:** Çağrılan araç sayısı ve türleri kaydedilir.
- **Alt-ajan sayısı:** Dinamik olarak başlatılan alt-ajan adedi raporlanır.
- **Token tüketimi:** API yanıtlarından alınan giriş/çıkış token sayıları biriktirilir.

Kısıt: Token telemetrisi, mevcut sürümde yalnızca eğitim içeriği üretimi fazında güvenilir biçimde raporlanabilmektedir. Bilgi tabanı oluşturma fazında API sağlayıcısı bazı durumlarda token sayacını döndürmediğinden, bu faz için tüketim kayıtları eksik kalabilmektedir.

4.1 Uygulama Tanıtımı

Geliştirilen sistem, bir yazılım projesini otomatik olarak inceleyip öğretici eğitim içeriği üreten ve yürütüm sırasında ilerleme bilgisi ile ölçüm kayıtlarını üreten bütünleşik bir çalışma akışı sunmaktadır. Uygulama komut satırından başlatılmakta; hedef yazılım sisteminin konumu ve yürütüm tercihleri kullanıcı tarafından belirlenmektedir. Çalışma boyunca yürütüm adımlarına ilişkin günlük kayıtları ve yapılandırılmış ölçüm verileri otomatik olarak üretilmektedir.

Bu bölümde sunulan karşılaştırma, aynı çekirdek ajan altyapısı ve aynı eğitim içeriği üretim araç takımı altında iki farklı koşulun etkisini incelemeyi amaçlamaktadır. İki koşul arasındaki temel fark; bilgi tabanı oluşturma aşamasının yürütülüp yürütülmemesi ve buna bağlı olarak istemlerin (ajan yönergelerinin) farklılaştırılmasıdır:

- **Deep Agent:** Yürütüm iki aşamalıdır. İlk aşamada yönetici ajan, görevi alt görevlere bölerek alt ajanlar aracılığıyla yazılım sisteminin bölümlerini izole biçimde inceler ve elde edilen bulguları kalıcı bir bilgi tabanında birleştirir. İkinci aşamada eğitim içeriği üretimi, bilgi tabanını birincil başvuru kaynağı olarak kullanacak biçimde tasarlanmış istemlerle yürütülür. Bu koşulda istemler; görev bölümleme stratejisi, pedagojik akış gereksinimleri, dosya yapısı kısıtları ve kalite kontrol yönergeleri içerecek biçimde kapsamlı olarak özelleştirilmiştir.
- **Baseline:** Yürütüm tek aşamalıdır. Bilgi tabanı oluşturma adımı çalıştırılmadan doğrudan eğitim içeriği üretimine geçilir. Bu koşulda istem tasarımı minimal düzeyde tutulmuştur: yalnızca dosya okuma yönergesi ve araç açıklamaları sağlanmaktadır. Özelleştirilmiş görev bölümleme, pedagojik yapılandırma veya kalite kontrol istemleri bulunmamaktadır.

Her iki koşulda da amaç, aynı giriş altında benzer nitelikte öğretici çıktı üretmek ve yürütüm süresi ile araç etkileşimini ölçerek karşılaştırmalı değerlendirmeye imkân

sağlamaktır. Deneysel koşulların yürütümünde temel üretim modeli olarak Google Cloud üzerinden **Gemini 2.5 Pro** kullanılmıştır. Üretilen eğitim dokümanları ile deneysel ölçüm özetleri, yürütüm sonunda otomatik olarak kaydedilmektedir.

4.2 Test Sonuçları

Sistemin başarımı, *Sistem Analizi* bölümünde belirlenen gereksinimler doğrultusunda; farklı büyüklük ve mimari özellikler taşıyan beş yazılım projesi üzerinde ölçülmüştür. Deneyler aynı yürütme ortamında iki koşul altında gerçekleştirilmiştir: (i) **Baseline** (bilgi tabanı oluşturma adımı olmaksızın doğrudan eğitim içeriği üretimi) ve (ii) **Deep Agent** (önce bilgi tabanı oluşturma, ardından bu bilgi tabanına dayalı eğitim içeriği üretimi). Her iki koşulda da alt-ajanlar sıralı (ardışık) olarak yürütülmüştür; yani bir alt-ajan görevi tamamlamadan bir sonraki başlatılmamıştır. Aynı ajan tipleri ve aynı eğitim içeriği üretim araç takımı kullanılmış; karşılaştırma, istem tasarımının ve bilgi tabanı kullanımının çıktılarına etkisini incelemeyi hedeflemiştir. Bu kapsamda çalışma süresi ve araç etkileşimi gibi göstergeler kaydedilmiştir. Bu çalışma kapsamında kullanılan ölçüm çıktıları, yürütme sırasında otomatik olarak üretilen deney raporlarından elde edilmiştir. Token kullanımı izlenmiş olmakla birlikte, bazı koşullarda eksik veya tutarsız olabildiğinden karşılaştırmalı nicel değerlendirmede birincil göstergeler olarak **çalışma süresi** ve **araç etkileşimi** esas alınmıştır (bkz. Alt Bölüm 4.2.2).

4.2.1 Çalışma Süresi ve Araç Kullanımı (Metrik Özeti)

Bu çalışmada beş açık kaynaklı yazılım projesi değerlendirilmiştir: deepagents [7], toon [8], RAG-Anything [9], Agent-Lightning [10] ve instructor [11]. Ölçüm özetleri okunabilirliği artırmak amacıyla iki ayrı tabloda raporlanmıştır.

Tablo 4.1 Deep Agent koşulunun çalışma süresi, araç etkileşimi ve iki aşamalı yürütüm göstergeleri. Veriler Aralık 2025 – Ocak 2026 yürütümlerinden derlenmiştir.

Yazılım Projesi	Toplam süre (sn) (Deep Agent)	Araç çağrısı (Deep Agent)	Alt-ajan (bilgi tabanı)	Alt-ajan (eğitim)	KB süresi (sn)	Eğitim süresi (sn)
deepagents	1254.7	80	6	6	724.9	529.8
toon	1264.3	80	7	6	849.9	414.4
RAG-Anything	903.4	67	4	7	362.6	540.8
Agent-Lightning	1320.9	101	6	7	700.0	621.0
instructor	1409.9	95	6	7	669.9	740.0

Analiz: Tablolar birlikte incelendiğinde, Deep Agent’in toplam süre ve araç çağrısı açısından Baseline’a kıyasla daha yüksek bir yürütüm maliyeti oluşturduğu görülmektedir. Bu farkın temel nedeni, Deep Agent’te bilgi tabanı oluşturma aşamasının ek bir işlem adımı olarak yürütülmesidir.

Tablo 4.2 Baseline koşulunun çalışma süresi, araç etkileşimi ve alt-ajan sayısı göstergeleri. Veriler Aralık 2025 – Ocak 2026 yürütümlerinden derlenmiştir.

Yazılım Projesi	Toplam süre (sn) (Baseline)	Araç çağırısı (Baseline)	Alt-ajan (Baseline)
deepagents	338.3	32	5
toon	325.9	26	5
RAG-Anything	280.8	22	3
Agent-Lightning	378.2	32	7
instructor	349.1	31	4

Buna karşın yürütümler sırasında bağlam sınırına ulaşıldığına işaret eden bir hata durumuna rastlanmamıştır. Bu bulgu, görevin alt ajanlar aracılığıyla ayrıştırılması ve üretim sürecinde ara çıktılardan sistematik biçimde yararlanılmasının bağlam yönetimi açısından uygulanabilir olduğuna işaret etmektedir.

4.2.2 Token Kullanımı

Token kullanımı, model yanıtlarında raporlanan kullanım bilgileri ve yürütüm sırasında derlenen özet kayıtlar üzerinden izlenmiştir. Bununla birlikte bu kayıtlar bazı koşullarda eksik veya tutarsız olabildiğinden, bu çalışmada token sayıları destekleyici bir gösterge olarak ele alınmış; ana karşılaştırma ölçütleri olarak çalışma süresi ve araç etkileşimi tercih edilmiştir.

Token Tüketimi ve Telemetri: 7 Ocak 2026 tarihli yürütümlerde, Google Cloud Gemini 2.5 Pro modelinde Baseline koşulda *deepagents* [7] projesi için 1.20–1.68 milyon token aralığında tüketim gözlemlenirken; Deep Agent koşulda aynı proje için 2.29 milyon token kullanılmıştır. *Toon* [8] projesi Deep Agent koşulda 1.23 milyon token tüketmiştir. Token telemetrisi, mevcut sürümde yalnızca **eğitim içeriği üretimi** fazında kaydedilebilmekte; **bilgi tabanı oluşturma** fazında kullanılan tokenler ise API sağlayıcısı tarafından henüz raporlanmamaktadır. Bu nedenle Deep Agent koşulunda raporlanan token değerleri, yalnızca ikinci fazı yansıtmaktadır.

4.3 Doğrulama

Sistemin ürettiği çıktıların kalitesi ve kullanıcı gereksinimlerini karşılama düzeyi, büyük dil modeli tabanlı değerlendirici (LLM-yargıç) yaklaşımıyla incelenmiştir [12].

4.3.1 LLM-yargıç Değerlendirmesi

Bu deneyde, dört farklı büyük dil modeli (**Claude 4.5 Sonnet**, **GPT-OSS-120B**, **Grok Code Fast 1**, **Gemini 2.5 Pro**) LLM-yargıç yaklaşımıyla değerlendirici olarak kulla-

nılmıştır. Değerlendiricilere, her bir yazılım projesi için iki koşulda üretilen eğitim dokümanları birlikte sunulmuş; ortak bir puanlama yönergesi çerçevesinde aşağıdaki ölçütlere göre değerlendirme yapmaları istenmiştir:

1. **Doğruluk (Fidelity):** Üretilen eğitim içeriğinin kaynak kodla tutarlılığı değerlendirilmiştir. Dosya veya klasör adları, API imzaları, bileşen ilişkileri ve çalışma akışı gibi somut iddiaların kod tabanında karşılığının olup olmadığı; uydurma bileşen veya özellik önermelerinin ve çelişkili açıklamaların (halüsinasyon) varlığı dikkate alınmıştır. (1: belirgin yanlış veya uydurma; 3: kısmi doğruluk, küçük hatalar; 5: tutarlı ve doğrulanabilir.)
2. **Pedagojik Uygunluk:** Anlatımın öğrenme hedeflerine hizmet edecek şekilde yapılandırılması ölçülmüştür. Önkoşulların belirtilmesi, basitten zora akış, adım adım yönergeler, örneklerin öğreticiliği ve okuyucunun uygulayabileceği bir kurgunun bulunması esas alınmıştır. (1: dağınık/uygulanamaz; 3: kısmen izlenebilir; 5: sistematik ve öğretici.)
3. **Kapsam:** İçeriğin kod tabanının kritik bileşenlerini ne ölçüde kapsadığı değerlendirilmiştir. Mimari/iş hattı, temel modüller, veri akışı ve geliştirme açısından önemli kullanım senaryolarına değinme düzeyi; yalnızca yüzeysel özetle sınırlı kalıp kalmadığı incelenmiştir. (1: yüzeysel; 3: kısmi; 5: kritik noktaları kapsayan.)

Her ölçüt 1–5 aralığında puanlanmış; ayrıca yargıçlardan iki koşul çıktısı arasında genel tercih bildirmeleri istenmiştir.

Yargıç Model	Tercih edilen eğitim dokümanı türü	Doğruluk		Pedagoji		Kapsam	
		Baseline	Deep Agent	Baseline	Deep Agent	Baseline	Deep Agent
Claude 4.5 Sonnet	Deep Agent	3.80	5.00	3.20	5.00	3.00	5.00
GPT-OSS-120B	Deep Agent	4.00	5.00	4.00	5.00	4.00	5.00
Grok Code Fast 1	Deep Agent	3.80	5.00	3.00	5.00	3.00	5.00
Gemini 2.5 Pro	Deep Agent	3.00	5.00	3.00	5.00	3.00	5.00
Tüm yargıçlar (toplam)	Deep Agent	3.65	5.00	3.30	5.00	3.25	5.00

Tablo 4.3 LLM-yargıçların beş yazılım projesi üzerindeki ortalama kalite puanları (5 üzerinden) ve tercih edilen eğitim dokümanı türü.

Sonuç Yorumu: Tablo 4.3 incelendiğinde, değerlendiricilerin her bir yazılım projesi için **Deep Agent** çıktısını tercih ettiği görülmektedir.

- **Doğruluk:** Baseline'ın bazı koşullarda güncelliğini yitirmiş arayüzleri önermesi veya isimlendirmede tutarsızlık göstermesi gözlenmiştir. Deep Agent ise bilgi tabanından doğrulama yapabildiği için daha tutarlı ve yüksek doğruluklu çıktılar üretmiştir.

- **Pedagoji:** Deep Agent; temel kavramlardan başlayıp ileri seviye konulara doğru ilerleyen, yapılandırılmış bir öğrenme akışı sunmuştur. Baseline’da ise anlatımın yer yer kopuklaştığı ve öğretim akışının zayıfladığı gözlenmiştir.
- **Kapsam:** Deep Agent, bilgi tabanında biriken ara çıktılarından yararlanarak yazılım projesinin daha geniş bir bölümüne değinebilen bir anlatım üretmiştir. Baseline’da ise bazı projelerde kritik bileşenlerin yüzeysel geçildiği gözlenmiştir.

4.3.2 Uzman Değerlendirmesi (İnsan Tercihi)

LLM-yargıç yaklaşımı otomatik ve ölçeklenebilir bir değerlendirme yöntemi sunmakla birlikte, büyük dil modellerinin kendi halüsinasyon eğilimleri nedeniyle yanıltıcı sonuçlar üretme riski bulunmaktadır. Bu riski değerlendirmek ve LLM-yargıç çıktılarının tutarlılığını doğrulamak amacıyla, üretilen eğitim içerikleri programlama ve yapay zekâ kütüphanelerine aşina uzmanlar tarafından incelenmiştir.

İnceleme sürecinde uzmanlara, her proje için Baseline ve Deep Agent koşullarında üretilen iki eğitim dokümanı *hangi yaklaşım tarafından üretildiği belirtilmeden* (kör test) sunulmuştur. Uzmanlardan hem A/B tercihi yapmaları hem de Doğruluk, Pedagoji ve Kapsam ölçütlerinde 5 üzerinden puanlama yapmaları istenmiştir.

Sonuçlar incelendiğinde, uzman değerlendirmelerinin LLM-yargıç çıktılarıyla tutarlı olduğu gözlemlenmiştir: uzmanların büyük çoğunluğu Deep Agent çıktısını tercih etmiş ve her üç ölçütte Deep Agent’in Baseline’a kıyasla üstünlük sağladığını doğrulamıştır. Bu tutarlılık, LLM-yargıç değerlendirmesinin güvenilir bir proxy olarak kullanılabileceğini göstermektedir.

4.3.3 Gereksinim Uyumluluk Analizi

Projenin başlangıcında (bkz. Bölüm 2.2) tanımlanan işlevsel ve işlevsel olmayan gereksinimler güçlü başlıklar halinde yeniden gözden geçirilmiştir. Denetimler, bilgi tabanı kullanımıyla yürütülen Deep Agent mimarisinin bu gereksinim setini büyük ölçüde karşıladığını göstermiştir.

Yönetici ajan; alt ajanları oluşturma, görevleri izole çalışma alanlarına dağıtma ve çıktıları denetleme sorumluluklarını yerine getirmektedir. Alt ajanların kapsamı, görev türüne göre sınırlandırılmış; böylelikle hem görev bölünmesi hem de bağlam yönetimi gereksinimleri sağlanmıştır. Güvenlik tarafında, kısıtlı dosya sistemi erişimi ve izinli dizin listesi sayesinde yetkisiz dosya/klasör erişimi engellenmiş; ölçüm kayıtlarında bu kısıtların ihlal edildiğine dair bulguya rastlanmamıştır.

Bilgi tabanı oluřturma fazı, koddan ıkarılan gzlemleri normalize ederek eđitim ieriđi retimi ncesinde kullanılabilir hale getirmekte; lmler de bu ara ıktıları dođrulamaktadır. Ayrıca yrtm sresince toplanan lm verileri (sre, ara ađrısı, token telemetrisi) gereksinimlerde belirtilen izlenebilirlik hedefini desteklemektedir. Sonu olarak sistem, otonom đretici materyal retimi grevini kalite ve izlenebilirlik bakımından yerine getirirken; Deep Agent yaklařımı, bilgi tabanı olmayan Baseline seeneđine kıyasla gereksinim uyumunda aık stnlk sađlamaktadır.

5 SONUÇ VE TARTIŞMA

Bu çalışmada, büyük ve karmaşık kod tabanlarının anlaşılmasını hızlandırmak amacıyla alt-ajan mimarisine dayalı otonom bir yapay zekâ sistemi tasarlanmış ve prototipi gerçekleştirilmiştir. Sistem, farklı karakteristiklere sahip **beş açık kaynaklı yazılım sistemi** üzerinde değerlendirilmiş; sonuçlar nicel göstergeler ve nitel LLM-yargıç değerlendirmeleriyle analiz edilmiştir.

Proje kapsamında elde edilen temel sonuçlar ve tartışma bulguları aşağıda sunulmuştur:

5.1 Elde Edilen Kazanımlar

1. **Bağlam Yönetimi:** Geleneksel tek ajanlı yaklaşımların temel sınırlılıklarından biri olan bağlam penceresi kısıtı, iki aşamalı mimari ve görev ayrıştırma stratejisiyle pratikte yönetilebilir hâle getirilmiştir. Yönetici ajanın tüm dosyaları doğrudan tüketmesi yerine görevi alt-ajanalara dağıtması ve özet temsiller üzerinden ilerlemesi, büyük ölçekli yazılım sistemlerinde sürdürülebilir yürütümü desteklemiştir.
2. **Kalite ve Derinlik Artışı:** Karşılaştırmalı değerlendirmeler (Alt Bölüm 4.2), Deep Agent'in ek analiz aşaması nedeniyle çalışma süresi ve araç etkileşimini artırmasına karşın, üretilen eğitim içeriğinin kalitesinde belirgin bir üstünlük sağladığını göstermiştir. Beş farklı kod tabanı üzerinde yürütülen LLM-yargıç değerlendirmelerinde, kullanılan yargıç modellerinin tamamında tercih edilen çıktı türü Deep Agent eğitim dokümanı olmuştur (Tablo 4.3).
3. **Güvenli ve İzole Çalışma:** Kısıtlanmış dosya erişim modeli sayesinde, otonom bileşenlerin yalnızca izinli çalışma alanlarında işlem yapması sağlanmış; böylece yetkisiz dizin gezintisi ve veri kaybı riskleri azaltılmıştır.

5.2 Kısıtlar ve Maliyet

Sistemin sunduğu yüksek kalitenin bir bedeli olarak işlem süresinde artış gözlenmiştir. Deep Agent’te, bilgi tabanı üretimi ve eğitim içeriği üretimi birlikte ele alındığında toplam sürenin yaklaşık **15–24 dakika** aralığına çıkabildiği; Baseline’da ise sürenin **5–7 dakika** bandında kaldığı görülmüştür (Tablo 4.2 ve Tablo 4.1). Bu farkın temel nedeni, Deep Agent’in bilgi tabanı oluşturma aşamasını ek olarak yürütmesidir.

Token sayımı bazı koşullarda sağlayıcı telemetrisi nedeniyle eksik raporlanabildiğinden, maliyet tartışması bu çalışmada token sayıları üzerinden değil; süre ve araç etkileşimi göstergeleri üzerinden yürütülmüştür. Gelecek çalışmalarda, token sayımı tutarlı biçimde raporlanan yürütme ortamlarıyla token-temelli maliyet analizi ayrıca ele alınabilir.

Geçerlilik Tehditleri: Bu çalışmanın bazı sınırlılıkları bulunmaktadır. LLM-yargıç değerlendirmelerinde kullanılan modellerin örtük eğilimleri bulunabilir ve farklı yargıç aileleri farklı sonuçlar üretebilir. Beş yazılım projesinden oluşan örneklem, tüm açık kaynak ekosistemini temsil etmeyebilir. Değerlendirme rubriği ve istem tasarımının puanlar üzerindeki etkisi ayrıca incelenmemiştir. Token telemetrisi kısıtı nedeniyle, mevcut sürümde token tüketimi yalnızca eğitim içeriği üretimi fazında raporlanabilmektedir; bilgi tabanı oluşturma fazında API sağlayıcısı (Google Cloud) henüz token sayacını döndürmemektedir. Bu nedenle Deep Agent için raporlanan token değerleri, toplam tüketimin yalnızca ikinci fazını kapsamaktadır ve gerçek toplam tüketim daha yüksek olabilir.

5.3 Gelecek Çalışmalar

Mevcut prototipin başarısı, bu alanda yapılacak ileri çalışmalar için güçlü bir temel oluşturmaktadır. Bu çalışmada Deep Agent ve Baseline (bilgi tabanı olmadan yürütüm) karşılaştırması ile uzman incelemesi gerçekleştirilmiştir. Gelecek dönemde şu geliştirmelerin yapılması hedeflenmektedir:

- **Genişletilmiş Kullanıcı Çalışmaları:** Mevcut uzman incelemesinin ötesinde, yazılım geliştiriciler ve üniversite öğrencileriyle daha geniş ölçekli kullanıcı çalışmalarının yürütülmesi; böylece pedagojik faydanın farklı deneyim seviyelerindeki kullanıcılarla doğrulanması.
- **Çoklu Model Karşılaştırması:** Farklı LLM sağlayıcıları (OpenAI, Anthropic, açık kaynak modeller) üzerinde sistemin performansının karşılaştırmalı değerlendirilmesi.

- **Web Arayüzünün Olgunlaştırılması:** Üretilen öğretici içeriklerin gezilmesi ve düzenlenmesi için geliştirilen web arayüzünün kullanılabilirlik, performans ve deney çıktılarının sunumu açısından geliştirilmesi.

Sonuç olarak, elde edilen bulgular otonom ajanların yalnızca kod üretimi değil; aynı zamanda mevcut kod tabanını açıklama ve öğretim amaçlı yapılandırılmış içerik üretimi görevlerinde de etkili bir yardımcı bileşen olabileceğini göstermektedir.

A

YARGICI DEĞERLENDİRME ÖRNEKLERİ

Bu ekte, LLM-yargıç değerlendirmesinde kullanılan istem örneği, eğitim içeriği örneği ve yargıç çıktısı sunulmaktadır.

A.1 Yargıç İstemi (Judge Prompt)

Aşağıda değerlendirme sürecinde yargıç modellere verilen istem örneği yer almaktadır:

Sen, yazılım kütüphaneleri için üretilen eğitim dokümanlarını değerlendiren uzman bir yargıçsın. Sana iki eğitim doküman serisi sunulacak: Seri A ve Seri B. Her iki seri de aynı kod tabanı için üretilmiştir.

Görevin, kod tabanını inceleyerek her iki serinin doğruluğunu, pedagojik uygunluğunu ve kapsamını değerlendirmektir. Değerlendirme kriterleri:

- 1. Doğruluk (Fidelity): Kod örneklerinin ve API referanslarının kaynak kodla tutarlılığı. Hatalı import yolları, yanlış fonksiyon isimleri veya uydurma bileşenler var mı?*
- 2. Pedagoji: Eğitim materyali basitten karmaşığa doğru mantıklı bir akış izliyor mu? Önkoşullar belirtilmiş mi? Adım adım yönergeler var mı?*
- 3. Kapsam: İçerik kod tabanının kritik bileşenlerini (temel API'ler, yapılandırma, CLI kullanımı vb.) kapsıyor mu?*

Her ölçüt için 1-5 arası puan ver ve hangi serinin genel olarak üstün olduğunu belirt.

A.2 Örnek Eğitim İçeriği Karşılaştırması

A.2.1 Baseline Çıktısı (Özet)

instructor kütüphanesi, yapılandırılmış LLM çıktıları elde etmenizi sağlar. Temel kullanım için `instructor.patch()` fonksiyonunu çağırmanız yeterlidir...

[Not: Baseline çıktısında dosya yapısı detayları eksik, API referansları yüzeysel]

A.2.2 Deep Agent Çıktısı (Özet)

instructor kütüphanesi, Pydantic modelleri kullanarak LLM yanıtlarını yapılandırmanızı sağlar.

Kurulum: `pip install instructor`

Temel Kullanım:

```
import instructor
from pydantic import BaseModel
from openai import OpenAI

class User(BaseModel):
    name: str
    age: int

client = instructor.patch(OpenAI())
user = client.chat.completions.create(
    model="gpt-4",
    response_model=User,
    messages=[{"role": "user", "content": "John is 25"}]
)
```

[Not: Deep Agent çıktısı doğru import yolları, çalışan kod örnekleri ve adım adım açıklamalar içermektedir]

A.3 Yargıç Değerlendirme Çıktısı (Örnek)

Aşağıda GPT-OSS-120B yargıcının instructor projesi için verdiği değerlendirme özeti yer almaktadır:

Tercih: Seri B (Deep Agent)

Puanlar:

- Doğruluk: Seri A: 4/5, Seri B: 5/5
- Pedagoji: Seri A: 4/5, Seri B: 5/5
- Kapsam: Seri A: 4/5, Seri B: 5/5

Gerekçe: Her iki seri de temel kavramları doğru açıklamaktadır. Ancak Seri B (Deep Agent), doğru import yolları kullanması, streaming ve hata yönetimi gibi ileri konuları kapsamaması ve pedagojik olarak daha yapılandırılmış bir akış sunması nedeniyle üstün bulunmuştur. Seri A'da bazı API referanslarının yüzeysel kaldığı ve ileri düzey özelliklerin eksik olduğu gözlemlenmiştir.

Referanslar

- [1] S. Yao ve diğ., “ReAct: Synergizing Reasoning and Acting in Language Models”, içinde *International Conference on Learning Representations (ICLR)*, 2023.
- [2] L. Wang ve diğ., “Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models”, *arXiv preprint arXiv:2305.04091*, 2023.
- [3] B. Xu, Z. Peng, B. Lei, S. Mukherjee, Y. Liu ve D. Xu, “ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models”, *arXiv preprint arXiv:2305.18323*, 2023.
- [4] J. Wang, J. Wang, B. Athiwaratkun, C. Zhang ve J. Zou, “Mixture-of-Agents Enhances Large Language Model Capabilities”, *arXiv preprint arXiv:2406.04692*, 2024.
- [5] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin ve B. Ghanem, “CAMEL: Communicative Agents for Mind Exploration of Large Language Model Society”, içinde *NeurIPS*, 2023.
- [6] Hugging Face, *smolagents: A Lightweight Library to Build Agents*, GitHub repository, 2024. erişim tarihi 8 Ocak 2026. erişim adresi: <https://github.com/huggingface/smolagents>.
- [7] LangChain, *DeepAgents*, GitHub repository, 2024. erişim tarihi 7 Ocak 2026. erişim adresi: <https://github.com/langchain-ai/deepagents>.
- [8] toon-format, *Toon*, GitHub repository, 2024. erişim tarihi 7 Ocak 2026. erişim adresi: <https://github.com/toon-format/toon>.
- [9] HKUDS, *RAG-Anything*, GitHub repository, 2024. erişim tarihi 7 Ocak 2026. erişim adresi: <https://github.com/HKUDS/RAG-Anything>.
- [10] Microsoft, *Agent Lightning*, GitHub repository, 2024. erişim tarihi 7 Ocak 2026. erişim adresi: <https://github.com/microsoft/agent-lightning>.
- [11] jxnl, *Instructor*, GitHub repository, 2023. erişim tarihi 7 Ocak 2026. erişim adresi: <https://github.com/jxnl/instructor>.
- [12] M. Zhuge ve diğ., “Agent-as-a-Judge: Evaluate Agents with Agents”, *arXiv preprint arXiv:2410.10934*, 2024.

BİRİNCİ ÜYE

İsim-Soyisim: BORA İLCİ

Doğum Tarihi ve Yeri: 04.10.2003, Bursa

E-mail: bora.ilci@std.yildiz.edu.tr

Telefon: 0541 200 31 04

Staj Tecrübeleri: Orion Innovation Türkiye Ar-Ge Departmanı

İKİNCİ ÜYE

İsim-Soyisim: KAAN EMRE KARA

Doğum Tarihi ve Yeri: 24.09.2002, İstanbul

E-mail: emre.kara@std.yildiz.edu.tr

Telefon: 0530 776 8079

Staj Tecrübeleri: Nef Solution Yazılım Departmanı

Proje Sistem Bilgileri

Sistem ve Yazılım: Windows, macOS veya Linux; Python tabanlı uygulama

Gerekli RAM: 8 GB

Gerekli Disk: 1 GB